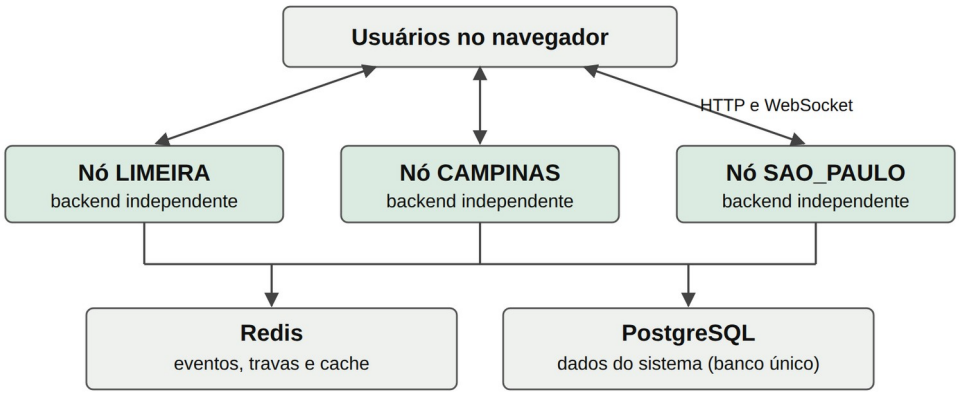
8. Relatório com as linhas de código do programa

Trechos do Compartilhaê ligados a sistemas distribuídos, copiados do repositório. Linhas que apenas registram log foram retiradas, e "// ..." indica outras omissões. Telas e saídas são de uma execução real com três nós, em 08/11/2026.

8.1. Múltiplos nós

Figura 1 — Visão geral da arquitetura



Listagem 1 — Identificação de um nó (docker-compose.yml)

```
NODE_ID: CAMPINAS
NODE_NAME: Node Campinas
NODE_CITY: Campinas
NODE_STATE: SP
```

8.2. Comunicação entre os nós

Listagem 2 — Formato da mensagem entre os nós (sync.service.ts)

```
export interface SyncEventPayload {
    eventId: string;
    originNode: string;
    sequence: string;
    eventType: string;
    entityId: string;
    payload: unknown;
    timestamp: string;
}
```

Listagem 3 — Publicação do evento (sync.service.ts)

```
async publishEvent(eventType: string, entityId: string, payload: unknown, tx?:
      Prisma.TransactionClient) {
    const client = tx ?? this.prisma;
    const event = await client.syncEvent.create({
        data: {
            eventId: randomUUID(),
            originNode: this.nodeId,
            eventType,
            entityId,
            payload: payload as object,
            status: SyncEventStatus.PENDING,
            correlationId: requestContext.getCorrelationId(),
        },
    });
    // ...
    if (tx) return;

    await this.attemptDelivery(event.id, event.eventId, {
    // ...
    });
}
```

Listagem 4 — Envio do evento para o Redis Stream (sync.service.ts)

```
private async attemptDelivery(rowId: string, eventId: string, message:
      SyncEventPayload) {
    try {
        await this.publisher.xadd(STREAM_KEY, '*', 'data', JSON.stringify(message));
        await this.prisma.syncEvent.update({
            where: { id: rowId },
            data: { status: SyncEventStatus.SYNCED, syncedAt: new Date() },
        });
        this.broadcastChange(message.eventType, message.entityId);
```

```
    } catch (err) {
        await this.prisma.syncEvent.update({
            where: { id: rowId },
            data: {
                status: SyncEventStatus.FAILED,
                retryCount: { increment: 1 },
                errorMessage: (err as Error).message,
            },
        });
    }
}
```

Listagem 5 — Leitura contínua do stream (sync.service.ts)

```
private async readLoop() {
    let cursor = (await this.reader.get(this.cursorKey)) ?? '$';

    while (this.running) {
        try {
            const result = await this.reader.xread('BLOCK', BLOCK_MS, 'STREAMS',
                 STREAM_KEY, cursor);
            if (!result) continue;

            const [, entries] = result[0];
            for (const [entryId, fields] of entries) {
                cursor = entryId;
                const dataIndex = fields.indexOf('data');
                if (dataIndex !== -1 && fields[dataIndex + 1]) {
                    this.handleIncoming(fields[dataIndex + 1]);
                }
            }
            await this.reader.set(this.cursorKey, cursor);
        } catch (err) {
            if (!this.running) return;
```

```
            await new Promise((resolve) => setTimeout(resolve, 2000));
        }
    }
}
```

Listagem 6 — Tratamento do evento recebido (sync.service.ts)

```
private handleIncoming(raw: string) {
    let message: SyncEventPayload;
    try {
        message = JSON.parse(raw) as SyncEventPayload;
    } catch {
        return;
    }

    if (message.originNode === this.nodeId) {
        // Eco da própria publicação: já processado localmente, ignorar.
        return;
    }
```

8.3. Detecção de falhas

Listagem 7 — Intervalo do heartbeat e limite para OFFLINE (nodes.service.ts)

```
const HEARTBEAT_INTERVAL_MS = 15_000;
const OFFLINE_THRESHOLD_MS = 45_000;
```

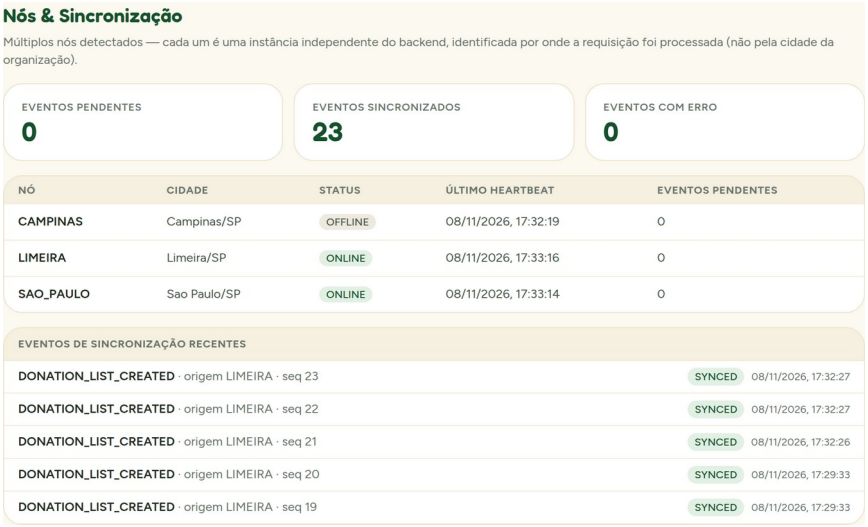
Listagem 8 — Envio do heartbeat (nodes.service.ts)

```
async onModuleInit() {
    await this.upsertSelf();
    this.heartbeatTimer = setInterval(
        () => void this.upsertSelf().catch((err: Error) => this.logger.error(err)),
        HEARTBEAT_INTERVAL_MS,
    );
}
```

Listagem 9 — Cálculo do estado do nó (nodes.service.ts)

```
return nodes.map((node) => {
    const lastHeartbeatMs = node.lastHeartbeat?.getTime() ?? 0;
    const isStale = now - lastHeartbeatMs > OFFLINE_THRESHOLD_MS;
    return {
        ...node,
        status: isStale ? NodeStatus.OFFLINE : node.status,
        pendingEvents: pendingByNode.get(node.code) ?? 0,
    };
});
```

Figura 2 — Painel com o nó CAMPINAS parado



8.4. Tolerância a falhas

Listagem 10 — Reenvio de eventos não entregues (sync.service.ts)

```
@Cron(CronExpression.EVERY_30_SECONDS)
async retryFailedEvents() {
    const pending = await this.prisma.syncEvent.findMany({
        where: {
            originNode: this.nodeId,
            status: { in: [SyncEventStatus.PENDING, SyncEventStatus.FAILED] },
            retryCount: { lt: MAX_RETRIES },
        },
```

```
        take: 50,
    });

    for (const event of pending) {
        await this.attemptDelivery(event.id, event.eventId, {
        // ...
        });
    }
// ...
}
```

Saída 1 — Log de CAMPINAS ao voltar da queda

```
Recebeu DONATION_LIST_CREATED do nó LIMEIRA, sequência 21
Recebeu DONATION_LIST_CREATED do nó LIMEIRA, sequência 22
Recebeu DONATION_LIST_CREATED do nó LIMEIRA, sequência 23
```

8.5. Concorrência

Listagem 11 — Trava por ONG e doação (reservations.service.ts)

```
const lockKey = `reservation:${donationList.id}:${ngoOrganization.id}`;
await tx.$queryRaw`SELECT pg_advisory_xact_lock(hashtext(${lockKey}))::text`;
```

Listagem 12 — Trava dos itens com FOR UPDATE (reservations.service.ts)

```
const rows = await tx.$queryRaw<LockedItemRow[]>(
    Prisma.sql`SELECT id, "foodName", unit, "remainingQuantity"::text AS
          "remainingQuantity", status,
                    "pickupDeadline"
                    FROM donation_items
                    WHERE id = ANY(${itemIds})
                    AND "donationListId" = ${donationList.id}
                    AND status = 'AVAILABLE'
                    ORDER BY id
                    FOR UPDATE`,
);
```

Listagem 13 — Evento gravado na mesma transação (reservations.service.ts)

```
// Evento no outbox dentro da mesma transação: se o processo cair entre o
// commit e a publicação, o cron de reenvio ainda encontra a linha.
await this.syncService.publishEvent('DONATION_RESERVED', donationList.id,
      reservation, tx);
```

Saída 2 — Reservas simultâneas (201 = criada; 409 = recusada)

```
Rodada 1: ONG Alfa em LIMEIRA = 201; ONG Beta em CAMPINAS = 409
Rodada 2: ONG Alfa em LIMEIRA = 409; ONG Beta em CAMPINAS = 201
Rodada 3: ONG Alfa em LIMEIRA = 201; ONG Beta em CAMPINAS = 409
Rodada 4: ONG Alfa em LIMEIRA = 201; ONG Beta em CAMPINAS = 409
Rodada 5: ONG Alfa em LIMEIRA = 201; ONG Beta em CAMPINAS = 409
```

8.6. Coordenação entre os nós

Listagem 14 — Trava distribuída no Redis (leader-lock.service.ts)

```
async runExclusively(
    jobName: string,
    task: () => Promise<void>,
    ttlSeconds = DEFAULT_TTL_SECONDS,
): Promise<boolean> {
    const key = `foodshare:job-lock:${jobName}`;
    const acquired = await this.redis.set(key, process.pid.toString(), 'EX', ttlSeconds,
         'NX');
    if (!acquired) return false;

    try {
        await task();
        return true;
    } catch (err) {
        return false;
    } finally {
        await this.redis.del(key).catch(() => undefined);
```

```
    }
}
```

Listagem 15 — Uso da trava em uma tarefa agendada (reservations-expiration.job.ts)

```
async expireAbandonedReservations() {
    await this.leaderLock.runExclusively('reservations-expiration', () => this.run());
}
```

8.7. Tempo real entre os nós

Listagem 16 — Ligação do Socket.IO ao Redis (redis-io.adapter.ts)

```
createIOServer(port: number, options?: ServerOptions): Server {
    // `IoAdapter.createIOServer` é declarado como `any`: sem este tipo, tudo que
    // encosta no servidor daqui para baixo deixa de ser verificado.
    const server = super.createIOServer(port, options) as Server;
    const pubClient = new Redis(this.redisUrl);
    const subClient = pubClient.duplicate();
    server.adapter(createAdapter(pubClient, subClient));
    return server;
}
```

Listagem 17 — Difusão da mudança (realtime.gateway.ts)

```
broadcastDataChanged(eventType: string, entityId: string) {
    this.server.emit('data:changed', { eventType, entityId });
}
```

Listagem 18 — Consulta de presença em todos os nós (realtime.gateway.ts)

```
async findUserIdsViewingConversation(conversationId: string):
      Promise<Set<string>> {
    const sockets = await this.server.in(`conv:${conversationId}`).fetchSockets();
    const userIds = sockets
        .map((socket) => (socket.data as Partial<SocketData>).userId)
```

```
        .filter((userId): userId is string => !!userId);
    return new Set(userIds);
}
```

Listagem 19 — Reação do cliente (live-updates.ts)

```
const unsubscribe = onSocketEvent<DataChangedPayload>('data:changed', () => {
    clearTimeout(timer);
    timer = setTimeout(() => queryClient.invalidateQueries(), 250);
});
```

8.8. Serviços externos

Listagem 20 — Consulta de CNPJ com três provedores (cnpj.service.ts)

```
async lookup(cnpjDigits: string): Promise<CnpjLookupResult> {
    const cacheKey = `foodshare:cnpj-cache:${cnpjDigits}`;
        // ...
        const cached = await this.redis.get(cacheKey);
        if (cached) return JSON.parse(cached) as CnpjLookupResult;
    // ...
    for (const provider of this.providers()) {
        try {
            const { data } = await firstValueFrom(
                this.http.get<unknown>(provider.url(cnpjDigits), {
                    timeout,
                    headers: { Accept: 'application/json', 'User-Agent': 'FoodShare/1.0' },
                }),
            );
            const result = provider.map(data);

            const ttl = this.config.get<number>('cnpjApi.cacheTtlSeconds') ?? 86400;
            await this.redis.set(cacheKey, JSON.stringify(result), 'EX', ttl).catch(() =>
                  undefined);

            return result;
```

```
        } catch (err) {
            // ...
        }
    }

    return { found: false };
}
```